Java
Exception Handling

A Beginner's Guide

CSE Department · UITS

1. What is an Exception?

An **exception** is a run-time error that occurs while a Java program is executing. The word "exception" stands for an **exceptional event** — something unexpected that interrupts the normal flow of the program.

Common reasons an exception can occur:

→ A user entered invalid data (e.g. typing letters where a number is expected)

→ A file that needs to be opened cannot be found

→ A network connection is lost during communication

→ The JVM runs out of memory

■ **Remember:**

If an exception is NOT handled, the Java interpreter displays the error and **terminates the program**. Exception handling lets the program survive and continue.

2. What is Exception Handling?

Exception handling is the process of detecting an error, catching it, and taking appropriate action so the program can continue running instead of crashing.

Java manages exceptions using **5 keywords**:

Keyword	What it does
`try`	Wraps the code that might throw an exception.
`catch`	Catches and handles the exception if one is thrown.
`throw`	Manually throws an exception object.
`throws`	Declares that a method might throw a certain exception.
`finally`	Code that always runs, whether or not an exception occurred.

3. General Form (Syntax)

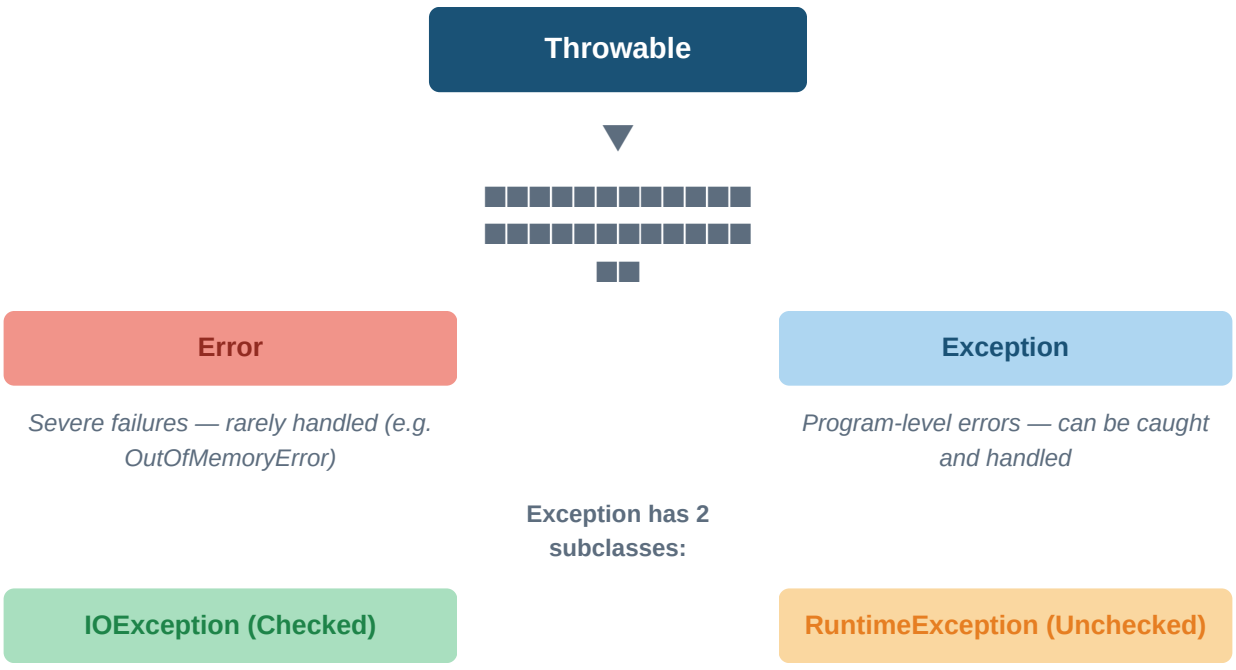
The basic structure of exception handling in Java looks like this:

Syntax — try / catch / finally

```
try {
// code that might cause an error
}
catch (ExceptionType1 e1) {
// handle ExceptionType1
}
catch (ExceptionType2 e2) {
// handle ExceptionType2
}
finally {
// always runs — cleanup code goes here
}
```

4. Exception Hierarchy

All exception classes in Java are subtypes of **java.lang.Exception**. The full hierarchy looks like this:



Checked vs Unchecked Exceptions

Type	Description
`Checked (IOException)`	Must be declared in the method's throws list. Cannot be ignored at compile time. Example: FileNotFoundException.
`Unchecked (RuntimeException)`	Do not need to be declared. Ignored by the compiler. Example: ArithmeticException, NullPointerException.
`Error`	Severe JVM-level failures. Not normally handled in code. Example: OutOfMemoryError, StackOverflowError.

5. Common Java Exceptions

Unchecked (RuntimeException) Subclasses

Exception	Meaning
`ArithmeticException`	Math error — e.g. divide by zero
`ArrayIndexOutOfBoundsException`	Array index is out-of-bounds
`NullPointerException`	Invalid use of a null reference
`NumberFormatException`	Invalid conversion of String to number
`ClassCastException`	Invalid object cast
`StringIndexOutOfBounds`	Accessing a character outside String length

Checked (IOException) Subclasses

Exception	Meaning
`ClassNotFoundException`	Class not found
`CloneNotSupportedException`	Attempt to clone an object that doesn't implement Cloneable
`IllegalAccessException`	Access to a class is denied
`InstantiationException`	Attempt to create object of an abstract class or interface
`InterruptedException`	One thread interrupted by another thread
`NoSuchMethodException`	A requested method does not exist

6. Code Examples

Example 1 — Basic try / catch

This example shows how catching an ArithmeticException (divide by zero) lets the program keep running and print the value of y.

Example 1 — try / catch (ArithmeticException)

```
public class TC_Demo {
public static void main(String[] args) {
int a = 10, b = 5, c = 5;
int x, y;
try {
x = a / (b - c); // b-c = 0 → divide by zero!
}
catch (ArithmeticException e) {
System.out.println("Divide by zero");
}
y = a / (b + c); // b+c = 10 → y = 1
System.out.println("y = " + y);
}
}
```

Output:

```
Divide by zero
y = 1
```

■ Tip:

The program did NOT stop when the exception occurred. It caught the error, printed the message, and continued to compute y = 1.

Example 2 — Multiple catch blocks

You can have several catch blocks for one try block — Java checks them top to bottom like cases in a switch. Always put the most specific exception first.

Example 2 — Multiple catch blocks

```
public class MultiCatch {
public static void main(String[] args) {
```

```
int a[] = {5, 10}; int b = 5;
try {
int x = a[2] / b - a[1]; // a[2] does not exist!
}
catch (ArithmeticException e) {
System.out.println("Divide by zero");
}
catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Array index error"); // this runs
}
catch (ArrayStoreException e) {
System.out.println("Wrong data type");
}
int y = a[1] / a[0];
System.out.println("y = " + y);
}
}
```

Output:

```
Array index error
y = 2
```

■ **Remember:**

When using multiple catch blocks, put **subclasses before superclasses**. If you put Exception first, it catches everything and the specific blocks below never run.

Example 3 — finally block

The `finally` block **always runs** — even if an exception occurs or even if no exception occurs. Use it for cleanup like closing files or connections.

Example 3 — finally block

```
public class Finally_Demo {
public static void main(String[] args) {
int num1=100, num2=50, num3=50;
int result1;
try {
result1 = num1 / (num2 - num3); // divide by zero
```

```
System.out.println("Result = " + result1);
}
catch (ArithmeticException g) {
System.out.println("Division by zero");
}
finally {
System.out.println("This is final"); // always runs
}
}
}
```

Output:

```
Division by zero
This is final
```

Example 4 — Creating your own exception (throw)

You can define and throw your own custom exceptions by extending the `Exception` class. This is useful when you want to enforce your own validation rules.

Example 4 — Custom exception with throw

```
import java.lang.Exception;

// Step 1: Create custom exception class
class MyException extends Exception {
MyException(String message) {
super(message);
}
}

// Step 2: Use it
class TestMyException {
public static void main(String[] args) {
int x = 5, y = 1000;
try {
float z = (float)x / (float)y;
if (z < 0.01) {
throw new MyException("Number is too small");
}
```

```
}
catch (MyException e) {
System.out.println("Caught MyException");
System.out.println(e.getMessage());
}
finally {
System.out.println("java2all.com");
}
}
}
```

Output:

```
Caught MyException
Number is too small
java2all.com
```

■ Tip:

Any class that extends Exception (or Throwable) can be thrown and caught. Use e.getMessage() to retrieve the message you passed to super().

7. throw vs throws

These two keywords look similar but serve very different purposes:

`throw`	Used inside a method to actually throw an exception object. Example: throw new ArithmeticException();
`throws`	Used in a method signature to declare it might throw an exception. Example: void myMethod() throws IOException { ... }

■ **Remember:**

throw actually fires the exception. **throws** is just a warning label on the method telling callers 'this might go wrong'. They are NOT interchangeable.

8. Quick Reference Card

`try { }`	Put risky code here. If it throws, jump to catch.
`catch (E e) { }`	Handle the exception. Print message, take action.
`finally { }`	Always runs. Use for cleanup (close files, etc).
`throw obj;`	Manually throw an exception object.
`throws E`	Tell callers this method might throw E.
`e.getMessage()`	Returns the error description string.
`e.printStackTrace()`	Prints full error trace to console (for debugging).

Exception handling makes your programs robust. Instead of crashing, they respond gracefully to the unexpected.